

SOLID

Five principles that make high cohesion and low coupling repeatable.

From Instinct to Principle

You already know the goal — high cohesion, low coupling, clean code. SOLID gives you five named, teachable moves to get there on purpose.

- SOLID is five object-oriented design principles (Robert C. Martin).
- Each one attacks a specific way that coupling and rigidity creep in.
- They are guidelines, not laws — apply them where change is likely.

The Five Principles

	Principle	In one line
S	Single Responsibility	One class, one reason to change
O	Open / Closed	Open to extension, closed to modification
L	Liskov Substitution	Subtypes must honour the base type's contract
I	Interface Segregation	Many small interfaces beat one fat one
D	Dependency Inversion	Depend on abstractions, not concretions

S — PRINCIPLE 01

Single Responsibility

A class should have one, and only one, reason to change.

One Class, One Reason to Change

'Responsibility' means a reason to change. If two different stakeholders can force edits to the same class, it has two responsibilities.

Smells

- A class named ...Manager or ...Utils
- Report + database + formatting in one
- Edits for unrelated reasons collide

Fix

- Separate the reasons to change
- One cohesive job per class
- Compose the jobs from outside

Split the Reasons to Change

Violates

```
class Invoice {  
    double total() {...}  
    void saveToDb() {...}    // persistence  
    String toHtml() {...}   // presentation  
}  
// 3 reasons to change in 1 class
```

Follows

```
class Invoice {  
    double total() {...}  
}  
class InvoiceRepository {    // persistence  
    void save(Invoice i) {...}  
}  
class InvoiceRenderer {      // presentation  
    String toHtml(Invoice i) {...}  
}
```

Now finance, storage and UI evolve independently — higher cohesion, lower coupling.

O — PRINCIPLE 02

Open / Closed

Open for extension, closed for modification.

Extend Without Editing

You should be able to add new behaviour by adding new code — not by editing code that already works and is already tested.

- Every edit to working code risks breaking it.
- A growing if/switch over 'types' is the classic violation.
- Introduce an abstraction; add new cases as new implementations.

The Type-Field Smell

int type + if-else

```
class Employee {  
    int type;    // 1=ENG 2=SALES 3=MGR  
    double pay() {  
        if (type == 1) return base();  
        if (type == 2) return base() + comm();  
        if (type == 3) return base() + bonus();  
        throw new IllegalStateException();  
    }  
}
```

one subclass per role

```
abstract class Employee {  
    abstract double pay();    // its own rule  
}  
class Engineer extends Employee {  
    double pay() { return base(); }  
}  
class Salesman extends Employee {  
    double pay() { return base() + comm(); }  
}  
// Manager, Contractor, ... each a new class
```

Add a role, edit the method — and every other switch on type. The fix is ‘replace conditional with polymorphism’: each type becomes a subclass. Open to new roles, closed to edits.

Add Behaviour by Adding Code

Closed to extension

```
double price(Item i) {  
    if (i.type == BOOK) return ...;  
    if (i.type == FOOD) return ...;  
    // new type? edit this method again  
}
```

Open to extension

```
interface Pricing { double price(Item i); }  
class BookPricing implements Pricing {...}  
class FoodPricing implements Pricing {...}  
// new type = new class, no edits here
```

We'll meet this again as the Strategy pattern in the next session.

L — PRINCIPLE 03

Liskov Substitution

Subtypes must be usable anywhere their base type is expected.

Subtypes Must Keep the Promise

If S is a subtype of T, you must be able to use an S wherever a T is expected — without surprises. Inheritance is a promise about behaviour.

Violation

- Square extends Rectangle, breaks `setWidth`
- Subtype throws where base didn't
- Subtype weakens the base's guarantees

Test

- Would every caller of the base still work?
- Same inputs → compatible outputs
- If not, it isn't really an 'is-a'

When 'is-a' Lies

Java*Square breaks Rectangle's contract*

```
class Rectangle { void setWidth(int w){} void setHeight(int h){} }

class Square extends Rectangle { // a Square 'is-a' Rectangle?
    void setWidth(int w) { super.setWidth(w); super.setHeight(w); }
    void setHeight(int h) { super.setWidth(h); super.setHeight(h); }
}

// A caller that sets width=5, height=4 expects area 20 – gets 16. Broken.
```

Prefer composition over forcing an inheritance that violates the contract.

I — PRINCIPLE 04

Interface Segregation

No client should depend on methods it does not use.

Many Small Interfaces

A 'fat' interface forces implementers to support methods they don't need — and couples clients to behaviour they never call.

- Split large interfaces along how clients actually use them.
- Implementers provide only what they truly support.
- Clients depend on the small slice they need — nothing more.

Split the Fat Interface

Fat interface

```
interface Machine {  
    void print();  
    void scan();  
    void fax();  
}  
// a simple printer must fake scan/fax
```

Segregated

```
interface Printer { void print(); }  
interface Scanner { void scan(); }  
interface Fax      { void fax(); }  
// implement only what you support  
class BasicPrinter implements Printer {...}
```


D — PRINCIPLE 05

Dependency Inversion

Depend on abstractions, not on concrete details.

Point Dependencies at Abstractions

High-level policy should not depend on low-level details. Both should depend on an abstraction — so details can change without touching policy.

Without DIP

- OrderService creates new MySqlConnection()
- Policy welded to a specific technology
- Can't test without the real database

With DIP

- OrderService depends on a Repository interface
- The concrete DB is injected from outside
- Swap DB or use a fake in tests freely

Inject the Abstraction

Depends on detail

```
class OrderService {  
    MySqlDb db = new MySqlDb();  
    // welded to MySQL forever  
}
```

Depends on abstraction

```
class OrderService {  
    final Repository repo;  
    OrderService(Repository r){ repo = r; }  
    // any Repository: MySQL, in-memory, fake  
}
```

This is the engine behind testability and Hexagonal architecture (Phase 2).

SOLID Is One Idea, Five Ways

Every principle is a tactic for the same strategy: keep things cohesive and loosely coupled so the system absorbs change.



S · I

Push cohesion up — small, focused units and interfaces.



O · L · D

Push coupling down — depend on stable abstractions.



All five

Together: a codebase you can extend without fear.

SOLID as a Review Lens

SOLID gives you precise language to steer and critique AI-generated code.

- Ask the AI to flag SRP or DIP violations in a diff before you accept it.
- Name the principle in your prompt: 'make this open/closed over payment types'.
- You judge whether the abstraction earns its keep — don't over-engineer.

REMEMBER THESE

Key Takeaways

- ✓ SRP: one class, one reason to change.
- ✓ OCP: extend by adding code, not editing working code.
- ✓ LSP: subtypes must honour the base type's contract.
- ✓ ISP: many small interfaces beat one fat one.
- ✓ DIP: depend on abstractions — the key to testability and clean architecture.

What's Next

MODULE 5 · SATURDAY 25 JULY

Design Patterns

with Akin Kaldıroğlu

Factory, Strategy and Proxy — proven solutions that put SOLID to work, so you stop reinventing the same designs.

Turn Principles into Moves

Apply SOLID as concrete refactoring moves on the ACME Orders code — starting with the god-service.

Refactoring course · SOLID

Repo github.com/kaldiroglu/refactoring-course-student-materials

IN THIS SESSION, LOOK AT

- ▶ Exercises/Ex03-ThreeMoves.md , Slides-PDF/Ch03-Principles.pdf
- ▶ .../service/OrderService.java (SRP)
- ▶ .../notify/EmailSender.java , SmsSender.java (DIP / OCP)

Questions?

Five principles. One goal: change without fear.